

Universidad ORT
Facultad de Ingeniería

Obligatorio 2 - Diseño de Aplicaciones 1
Ingeniería en Sistemas

Martina Barone - 302285

Inés Gayol - 281635

Ana Clara Vázquez - 282326

Profesores: Juan Ignacio Díaz, Santiago Pérez y Luis Eduardo Simón

2025

Link al repositorio de la entrega:

https://github.com/IngSoft-DA1/302285_281635_282326

Índice

1. Descripción general	4
2. Bug conocido	4
3. Diseño	5
4. Diagrama de paquetes	15
5. Diagramas de clases	16
6. Diagramas de interacción	16
7. Modelo de tablas	16
8. Principales decisiones de diseño tomadas	17
9. Criterios seguidos para asignar responsabilidades	18
10. Análisis de dependencias para el requerimiento: Resolución de conflictos en asignación de recursos a tareas	22
11. Cobertura de pruebas unitarias	23
12. Referencias bibliográficas.....	24
Anexo 1. Diagramas UML	25
Anexo 2. Diagramas de interacción	28
Anexo 3. Cobertura de pruebas unitarias	31
Anexo 4. Instalación	33
Anexo 5. Pruebas unitarias	34
Anexo 6. Datos de prueba	35

Descripción general

Este trabajo consiste en el desarrollo de un sistema de gestión de proyectos llamado **TaskTrackPro**, el cual permite la administración colaborativa de tareas, recursos y dependencias entre actividades dentro de un proyecto.

La aplicación fue construida utilizando el lenguaje C# bajo el framework .NET 8, implementada como una Blazor Server App, asegurando una experiencia web interactiva con renderizado en servidor. Se aplicó la metodología *Test Driven Development (TDD)* durante el diseño e implementación de la lógica del sistema, utilizando el framework de pruebas unitarias MSTest. Además, se siguieron los principios de *Clean Code*, con especial atención a la claridad, simplicidad y responsabilidad única en cada clase. Se favorecieron los principios y patrones de diseño *GRASP* y *S.O.L.I.D.* para una adecuada asignación de responsabilidades y un diseño mantenible.

Para las pruebas automatizadas, se utilizó EF Core In-Memory Database Provider, permitiendo validar la lógica de persistencia respetando los principios *F.I.R.S.T.*

La información del sistema persiste en una base de datos Azure SQL Edge, ejecutándose en un contenedor de Docker. El diseño contempla una solución de persistencia adecuada para el dominio del problema, utilizando Entity Framework Core con enfoque *Code First*. Los datos se almacenan progresivamente durante la operativa del sistema.

El desarrollo colaborativo se gestionó mediante GitHub, siguiendo estrictamente la metodología *Gitflow*.

Bug conocido

Al marcar una tarea con dependencias como completada, se muestra un mensaje de error que no resulta claro ni amigable para el usuario final. A pesar de la aparición de este mensaje, la operación se realiza correctamente: el estado de la tarea se actualiza como completado y el cambio se persiste adecuadamente en la base de datos. Este comportamiento no afecta la lógica ni la integridad del sistema, pero podría generar confusión durante el uso.

Diseño

Namespace	Clase	Responsabilidad
Dominio	Usuario	Representa a un usuario del sistema, es responsable de integridad de sus propios datos y comportamientos asociados a su rol en la aplicación.
Dominio	Proyecto	Representa un proyecto. Administra sus tareas, miembros, fechas clave, notificaciones, validaciones relacionadas y modifica sus datos. Además, gestiona el vínculo entre los usuarios y sus roles dentro del proyecto.
Dominio	Tarea	Representa una tarea, validando sus datos. Gestiona dependencias, miembros y recursos asignados. Controla los cambios de estado, calcula la fecha de finalización más temprana, determina si la tarea es crítica y permite modificar sus atributos.
Dominio	Recurso	Representa un recurso, validando sus datos. Gestiona su uso dentro de tareas, su posible asociación exclusiva a un proyecto y permite modificar sus atributos.

Dominio	RecursoNecesario	Representa un recurso requerido por una tarea, validando que sea válido y que su cantidad sea mayor a cero. Asocia el recurso con su cantidad necesaria.
Dominio	RangoDeUso	Representa un período en el que un recurso es utilizado. Valida que la fecha de inicio sea anterior a la de fin y que la cantidad de usos sea mayor a cero.
Dominio	Notificación	Representa una notificación del sistema. Asigna automáticamente un identificador único, valida que el mensaje no sea vacío y registra la fecha de creación.
Dominio	Dependencia	Representa una dependencia entre tareas. Valida que el tipo sea correcto y que la tarea asociada no sea nula. Permite comparar dependencias por tipo y tarea para evitar duplicaciones.
Dominio	EstadoTarea	Define los posibles estados que puede tener una tarea.
Excepciones	ExcepcionCaminoCritico	Representar errores específicos del Camino Crítico.

Excepciones	ExcepcionContraseña	Representar errores específicos de la contraseña.
Excepciones	ExcepcionDominio	Representar errores específicos del Dominio.
Excepciones	ExcepcionPermisos	Representar errores específicos de Permisos.
Excepciones	ExcepcionProyecto	Representar errores específicos de Proyectos.
Excepciones	ExcepcionRecurso	Representar errores específicos de Recursos.
Excepciones	ExcepcionTarea	Representar errores específicos de Tareas.
Excepciones	ExcepcionUsuarios	Representar errores específicos de Usuarios.
Excepciones	ExcepcionRangoDeUs o	Representar errores específicos de Rango De Uso.
Excepciones	ExcepcionConflicto	Representar errores específicos de Conflictos de Recursos.

Excepciones	ExcepcionExportador	Representar errores específicos de Exportador.
Excepciones.MensajeError	MensajeErrorDominio	Guarda los mensajes de error atribuidos a los errores del Dominio.
Excepciones.MensajeError	MensajeErrorServicios	Guarda los mensajes de error atribuidos a los errores de los Servicios.
Servicios.Gestores	GestorUsuarios	Gestiona el ciclo de vida de los usuarios, incluyendo su creación, eliminación, búsqueda y modificación de roles. Valida reglas de negocio relacionadas con permisos, contraseñas y asignaciones a proyectos. Además, administra notificaciones ante cambios relevantes en la cuenta de un usuario.
Servicios.Gestores	GestorProyectos	Administra la creación, eliminación y modificación de proyectos, incluyendo sus atributos principales como nombre, descripción y fecha de inicio. Gestiona los miembros del proyecto, permitiendo agregarlos, eliminarlos o cambiar al administrador. Asegura que se respeten las reglas del negocio mediante validaciones de permisos y estados, y permite consultar los proyectos

		relacionados a un usuario.
Servicios.Gestores	GestorTareas	Administra la creación, eliminación y actualización de tareas dentro de un proyecto. Permite modificar atributos como título, descripción, duración, fecha de inicio y estado. Gestiona dependencias, recursos y usuarios asignados a cada tarea. Valida los permisos y reglas de negocio antes de ejecutar cambios, recalcula el camino crítico ante modificaciones relevantes y notifica a los miembros del proyecto.
Servicios.Gestores	GestorRecursos	Administra la creación, eliminación y edición de recursos dentro del sistema. Permite modificar atributos como nombre, tipo y descripción, y asocia recursos exclusivos al proyecto correspondiente. Verifica que solo los usuarios con permisos adecuados puedan operar sobre cada recurso, diferenciando entre recursos generales y exclusivos. Además, controla si un recurso está en uso antes de permitir su eliminación o modificación, y notifica a los administradores involucrados ante cualquier cambio relevante.
Servicios.CaminoCriti	CaminoCritico	Calcula el camino crítico de un proyecto

co		a partir de sus tareas, considerando sus dependencias del tipo Fin-Inicio (FS) e Inicio-Inicio (SS). Ordena las tareas topológicamente para validar la ausencia de ciclos, determina las fechas más tempranas de inicio y fin, y calcula la holgura de cada tarea. Notifica a los miembros del proyecto si se modifica la fecha de finalización más temprana.
Utilidades	UtilidadesContrasena	Valida que una contraseña cumpla con los requisitos mínimos de seguridad definidos: longitud, inclusión de letras mayúsculas, minúsculas, números y caracteres especiales. Además, permite encriptar contraseñas utilizando el algoritmo BCrypt y generar contraseñas válidas de forma automática cumpliendo con los mismos criterios.
Utilidades	PermisosUsuarios	Centraliza la lógica de control de acceso y validación de permisos. Verifica si un usuario puede realizar acciones como administrar proyectos, modificar contraseñas, o cambiar estados de tareas según su rol y relaciones con el proyecto.
Servicios.Exportacion	ExportadorProyectosFactory	Implementa la lógica para crear exportadores de proyectos según el

		formato solicitado (CSV o JSON). Lanza una excepción si el formato no es soportado.
Servicios.Exportacion	ExportadorJson	Implementa la exportación de proyectos en formato JSON.
Servicios.Exportacion	ExportadorCsv	Implementa la exportación de proyectos en formato CSV.
Servicios.Notificaciones	MensajesNotificaciones	Guarda los mensajes de notificación del sistema.
Servicios.Notificaciones	Notificador	Se encarga de enviar notificaciones a usuarios, permitiendo notificar a un solo usuario o a varios.
IServicios	ICalculadorCaminoCritico	Define una funcionalidad para calcular el camino crítico de un proyecto, a partir de sus tareas y dependencias.
IServicios	IExportadorProyectos	Define la estructura para exportar proyectos en distintos formatos, incluyendo el nombre del formato, tipo de contenido, nombre del archivo y el método de exportación.

IServicios	IExportadorProyectos Factory	Define cómo crear una instancia del exportador de proyectos según el formato solicitado.
IServicios	INotificador	Define métodos para enviar notificaciones a uno o varios usuarios del sistema, para así poder comunicar acciones o eventos relevantes.
IServicios.IGestores	IGestorProyectos	Define las operaciones que implementa la clase GestorProyectos.
IServicios.IGestores	IGestorRecursos	Define las operaciones que implementa la clase GestorRecursos.
IServicios.IGestores	IGestorTareas	Define las operaciones que implementa la clase GestorTareas.
IServicios.IGestores	IGestorUsuarios	Define las operaciones que implementa la clase GestorUsuarios.
IRepositorios	IRepositorio	Define las operaciones básicas que debe implementar cualquier repositorio del sistema. Permite agregar un objeto, obtenerlo por su identificador, eliminarlo y recuperar la lista completa de objetos almacenados.

IRepositorios	IRepositorioUsuarios	Define las operaciones que cualquier repositorio de usuarios tiene que implementar: buscar por email y actualizar la contraseña encriptada, además de heredar los métodos genéricos del IRepositorio.
IRepositorios	IRepositorioProyectos	Define las operaciones que cualquier repositorio de proyectos tiene que implementar: ActualizarTarea, además de heredar los métodos genéricos del IRepositorio.
Repositorios	RepositorioUsuarios	Implementa los métodos de la interfaz de repositorio de usuarios.
Repositorios	RepositorioProyectos	Implementa los métodos de la interfaz de repositorio.
Repositorios	RepositorioRecursos	Implementa los métodos de la interfaz de repositorio.
Repositorios	SqlContext	Define las entidades (Usuario, Proyecto, Recurso) como tablas, aplica migraciones si la base de datos es relacional, y configura cómo se mapean las entidades al modelo relacional mediante métodos de configuración en

		OnModelCreating.
Repositorios.ConfiguracionesEntidades	ConfiguracionDependencia	Define cómo se mapea la entidad Dependencia a la base de datos usando Fluent API de Entity Framework Core.
Repositorios.ConfiguracionesEntidades	ConfiguracionNotificacion	Define cómo se mapea la entidad Notificacion a la base de datos usando Fluent API de Entity Framework Core.
Repositorios.ConfiguracionesEntidades	ConfiguracionProyecto	Define cómo se mapea la entidad Proyecto a la base de datos usando Fluent API de Entity Framework Core.
Repositorios.ConfiguracionesEntidades	ConfiguracionRecurso	Define cómo se mapea la entidad Recurso a la base de datos usando Fluent API de Entity Framework Core.
Repositorios.ConfiguracionesEntidades	ConfiguracionTarea	Define cómo se mapea la entidad Tarea a la base de datos usando Fluent API de Entity Framework Core.
Repositorios.ConfiguracionesEntidades	ConfiguracionUsuario	Define cómo se mapea la entidad Usuario a la base de datos usando Fluent API de Entity Framework Core.

Diagrama de paquetes (Ver anexo 1.1)

El sistema se organiza en paquetes que reflejan una arquitectura modular basada en la separación de responsabilidades. Esta estructura facilita la comprensión, el mantenimiento y la evolución del sistema, alineándose con los principios enseñados en el curso.

El paquete Dominio contiene las entidades centrales del modelo, como Usuario, Tarea, Recurso, Proyecto, entre otras. Cada clase encapsula sus propias validaciones, reglas de negocio y comportamientos asociados, promoviendo una alta cohesión. Este paquete no depende de ningún otro y es reutilizado por el resto de los componentes del sistema.

El paquete Servicios implementa la lógica de aplicación. Contiene los gestores que orquestan operaciones complejas sobre las entidades del dominio, y también contiene la lógica de Camino Crítico, Exportación y Notificaciones. Este paquete depende tanto de Dominio como de IRepositorios, IServicios, DTOs y Utilidades, permitiendo aplicar las reglas de negocio sin acoplarse a implementaciones concretas.

El paquete IRepositorios define las interfaces de persistencia que utilizan los servicios para interactuar con los datos. A su vez, estas interfaces son implementadas por el paquete Repositorios, que simula la capa de acceso a datos con almacenamiento en memoria. Esta separación permite sustituir fácilmente las implementaciones sin modificar la lógica de negocio.

El paquete Interfaz representa la capa de presentación construida con Blazor Server. Se comunica el paquete Controladores, que a su vez dependen de IServicios y DTOs para interactuar con los gestores de servicios. La interfaz está libre de lógica de negocio, la cual está completamente delegada a la capa de servicios.

El paquete DTOs facilita la transferencia de datos entre capas, desacoplando la interfaz de las entidades del dominio. También colabora en la validación de entrada y evita fugas de lógica innecesarias hacia la capa de presentación.

El paquete Utilidades agrupa clases de apoyo reutilizables, como las utilidades de contraseñas o verificación de permisos (PermisosUsuarios). Es utilizado por los Servicios para aplicar validaciones sin sobrecargar la lógica principal, centralizando las verificaciones en una sola clase.

El paquete Excepciones centraliza las clases de error y los mensajes asociados a las distintas validaciones del sistema, organizándose por tipo. Es utilizado principalmente por el Dominio, los Servicios y Utilidades.

Diagramas de clases

Dominio (Ver anexo 1.2)

Servicios (Ver anexo 1.3)

Utilidades (Ver anexo 1.4)

Repositorios (Ver anexo 1.5)

Diagramas de interacción (Ver anexo 2)

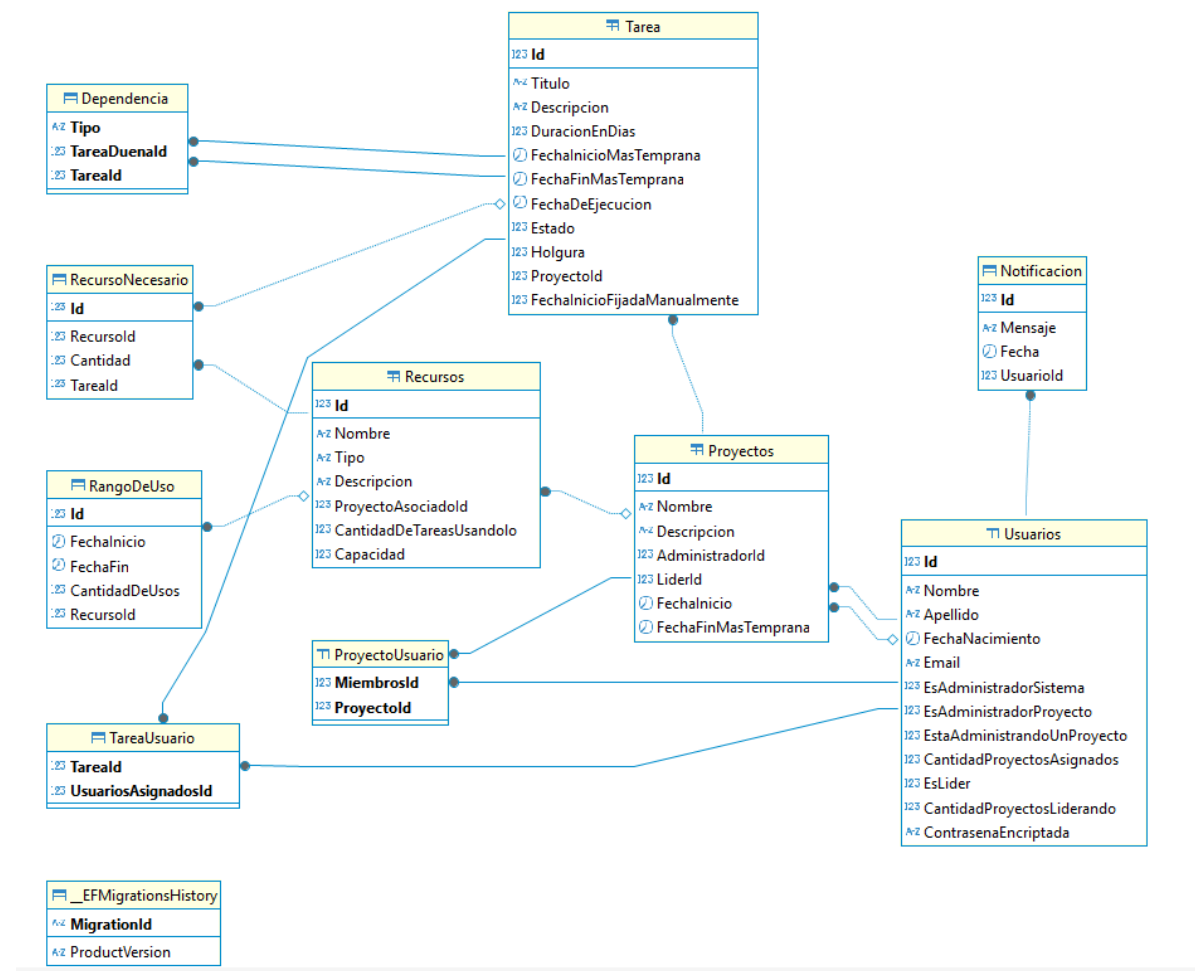
Modelo de tablas

A continuación se presenta el modelo de tablas de la base de datos tal como lo muestra DBeaver. El diseño de clases del sistema sigue una orientación a objetos, con entidades del dominio como Proyecto, Tarea, Usuario, Recurso, entre otras. Sin embargo, el modelo físico generado en la base de datos mediante Entity Framework Core incluye tablas adicionales o estructuras distintas debido a configuraciones realizadas mediante Fluent API, necesarias para controlar relaciones, restricciones y claves foráneas.

En base a la documentación de Microsoft¹, se pudieron modelar tablas que asocian entidades, en relaciones de uno a muchos o muchos a muchos. Por ejemplo, se modelaron las tablas TareaUsuario y ProyectoUsuario. A su vez, para las relaciones que EF Core no podía mapear por sí solo por falta de Foreign Keys (como IDs de entidades relacionadas), se especificaron como *Shadow Properties*, bajo la convención de *NombreClaseAsociadaId* (ej. ProyectoId, TareaId, cuando la clase solo tiene referencia al Proyecto o Tarea, no a sus ids)

¹ [Introducción a las relaciones - EF Core | Microsoft Learn](#)

Todas estas configuraciones con Fluent API, se encuentran en las clases Configuración en el paquete de Repositorios.



Principales decisiones de diseño tomadas

Los mecanismos generales, se especifican en la justificación de los diagramas de paquetes, presentada anteriormente. El Usuario interactúa con controladores, los Servicios se encargan de la lógica de negocio y los Repositorios se encargan del acceso a datos y persistencia.

Sin embargo, resulta pertinente en esta entrega, señalar decisiones de diseño en cuanto a la implementación y funcionamiento de la aplicación, más que nada referentes a las nuevas funcionalidades implementadas.

En primer lugar, se puntúan las decisiones referentes a la gestión de recursos:

- Solo se puede agregar un recurso a una tarea con su cantidad una vez. No se puede agregar varias veces el mismo recurso con distintas capacidades.
- Eliminar un recurso de la tarea elimina todos los usos del recurso, si en una tarea se usa 2 veces el mismo recurso, se eliminan los dos usos, no se puede disminuir a un uso.
- La resolución de conflictos en cuanto a recursos, no es automática. Se notifica al administrador con los cambios a realizar.
- Si una tarea tiene recursos reservados, no se le pueden modificar las dependencias ni su duración o fecha de inicio, ya que afectan en las fechas de la tarea y la reserva de recursos se puede ver afectada. Del mismo modo, si una tarea con recursos reservados no llegó a su fecha de fin, no se puede marcar completada antes de tiempo.
- El estado de las tareas solo refiere a dependencias/fechas. Nunca refiere a bloqueos por recursos.

En segundo lugar, se puntúan las decisiones referentes al rol líder:

- Solo un admin de su propio proyecto puede agregar un líder a su proyecto.
- Un mismo usuario puede ser líder de múltiples proyectos. Para representar esto, se agregó al badge de rol de Líder de proyecto la cantidad de proyectos que está liderando el usuario actualmente.

Criterios seguidos para asignar responsabilidades

Para asignar responsabilidades se siguieron los principios generales GRASP, a continuación se decide exponer los principales ejemplos del uso de dichos principios en el sistema implementado.

Principio creador

Se asignó a la clase Usuario la responsabilidad de crear instancias de notificaciones y agregarlas a su propia colección, a través del método `RecibirNotificación(mensaje)`. Esta decisión se fundamenta en que el usuario contiene, agrega y registra notificaciones, lo que

concuenda con el principio del Creador, al establecer que una clase debe ser responsable de crear objetos que contiene o utiliza directamente.

Asimismo, la clase Tarea es responsable de crear instancias de RecursoNecesario, dado que es la entidad que las contiene y gestiona.

Por último, la clase ExportadorProyectosFactory se encarga de crear instancias de exportadores concretos (como ExportadorCsv y ExportadorJson). Esta asignación se justifica en el hecho de que la fábrica posee los datos necesarios para la inicialización, particularmente el repositorio de proyectos, cumpliendo así con el principio del Creador al centralizar la lógica de instanciación de objetos relacionados.

Principio experto

Siguiendo el principio Experto, las operaciones vinculadas a la gestión de tareas fueron delegadas al repositorio de proyectos, ya que esta clase posee acceso directo a las tareas asociadas a cada proyecto y dispone del conocimiento necesario para manipularlas de manera eficiente.

Por otra parte, los gestores fueron diseñados como expertos en la coordinación de sus respectivas entidades de dominio. Estos componentes se responsabilizan de orquestar los casos de uso del sistema, colaborando con servicios especializados como el servicio de cálculo del camino crítico, el notificador y el validador de permisos. De este modo, cada clase concentra las responsabilidades para las cuales dispone de la información y el conocimiento adecuado, promoviendo una distribución coherente y mantenible de la lógica del sistema.

Bajo acoplamiento y alta cohesión

Uno de los principales objetivos a la hora de diseñar, fue mantener un bajo acoplamiento y alta cohesión entre clases asignando una única responsabilidad a cada una, respetando lo que propone GRASP. En la tabla expuesta en la sección Diseño de la documentación, se evidencia la asignación de responsabilidades únicas a cada clase.

Controlador

En el sistema, los eventos provenientes de la interfaz de usuario son manejados por controladores específicos por caso de uso, lo que permite adaptarse fácilmente a distintos

clientes. Estos controladores (ControladorTareas, ControladorRecursos, etc.) actúan como intermediarios entre la interfaz y la lógica de negocio, delegando las operaciones a los gestores correspondientes.

Además, se incluye un ControladorExportacion, que atiende la descarga solicitada por el líder del proyecto, delegando la creación del archivo a una fábrica.

Para mantener el principio de ocultamiento, todos los controladores operan exclusivamente con DTOs, siendo los gestores quienes realizan el mapeo desde y hacia las entidades del dominio.

Fabricación pura

Para la exportación de archivos se aplicó el principio de Fabricación Pura, creando clases como ExportadorCSV y ExportadorJSON, que no forman parte del dominio, pero centralizan la responsabilidad de generar archivos en distintos formatos. Estas clases implementan la interfaz IExportador, lo que permite extender el sistema sin modificar las clases existentes.

De manera similar, se diseñó la clase Notificador como una fabricación pura. Esta implementa INotificador y es utilizada por los gestores para enviar notificaciones a los usuarios, manteniendo bajo acoplamiento y evitando duplicación de lógica. Sus métodos NotificarUno y NotificarMuchos encapsulan el envío, delegando en el método RecibirNotificación del usuario.

Indirección

Se priorizó y favoreció el principio de Indirección mediante el uso de interfaces como INotificador, IRepositorioProyectos e ICalculadorCaminoCritico, que actúan como capas de abstracción entre gestores y sus implementaciones concretas. Esto disminuye el acoplamiento y mejora la extensibilidad. Además, se empleó la fábrica ExportadorProyectosFactory como otro mecanismo de indirección, permitiendo crear exportadores según el formato sin que el controlador conozca las clases específicas.

Variaciones protegidas

Al pensar el diseño se tuvo especial atención a los posibles cambios futuros del sistema, aplicando principios que facilitan la adaptación sin impactar el código existente.

Por ejemplo, la clase Notificador implementa una interfaz para anticipar distintas formas de envío de notificaciones. En particular, se prevé que en el futuro el sistema podría notificar eventos importantes, como el cambio de contraseña, vía correo electrónico u otros canales, como se menciona en el foro de requerimientos.

De forma similar, los exportadores implementan interfaces para permitir la incorporación de nuevos formatos de exportación sin modificar las clases actuales, asegurando así la extensibilidad del sistema ante futuras necesidades.

SOLID

No solo para asignar responsabilidades, sino que también para asegurar un buen diseño mantenible y extensible, se tuvieron en cuenta patrones SOLID.

Se consideró SRP (Single Responsibility Principle), al crear la utilidad PermisosUsuario para validar permisos y la clase Notificador para gestionar las notificaciones, evitando que los gestores asuman estas responsabilidades. Por ejemplo, GestorUsuarios no se ve afectado por cambios en la lógica de notificaciones o su forma de envío, manteniendo un único motivo para modificarse. Esto incrementa la cohesión y reduce el acoplamiento. Aunque la creación de nuevas clases aumenta la cantidad de componentes, se prioriza la clara separación de responsabilidades.

Por otro lado, se tuvo en cuenta OCP (Open/Closed Principle). Se cumple en casos gracias a clases como Notificador y los exportadores (ExportadorCSV, ExportadorJSON), que permiten extender funcionalidades (nuevos formatos de exportación o métodos de notificación) sin modificar el código existente, sino agregando nuevas implementaciones. Además, el uso de un enum para los estados de las tareas facilita la incorporación de nuevos estados centralizados sin alterar la lógica dispersa.

Para favorecer DIP (Dependency Inversion Principle), se crearon interfaces para los Servicios y Repositorios, y además se separaron las interfaces y sus implementaciones en distintos paquetes, por ejemplo: Servicios e IServicios, Repositorios e IRepositorios.

Asimismo, los controladores dependen de interfaces (IGestor) en lugar de implementaciones concretas, evitando acoplamientos directos a componentes de bajo nivel. De la misma forma,

los gestores dependen de interfaces de Repositorios o en algún caso de otro gestor/servicio, según las necesidades.

Por otro lado, como posible mejora surgió una forma de favorecer ISP (Interface Segregation Principle). Se sugiere dividir IGestorProyectos en dos; IGestorProyectosControlador que contiene únicamente los métodos que emplea el controlado e IGestorProyectos que hereda de IGestorProyectosControlador y agrega el resto de los métodos públicos de GestorProyectos. Esta separación evitaría que los controladores dependan de métodos que no usan, lo cual en el sistema actual ocurre en ese caso en concreto.

Análisis de dependencias para el requerimiento: Resolución de conflictos en asignación de recursos a tareas

La implementación de la funcionalidad relacionada con recursos requirió modelar a estos no solo como entidades con una capacidad diaria, sino también como elementos sujetos a disponibilidad temporal. Para ello se diseñó la clase RangoDeUso, que representa un intervalo de tiempo durante el cual se realiza una cierta cantidad de usos sobre un recurso determinado. Así, cada recurso mantiene una lista de rangos de uso, que es modificada dinámicamente por el gestor de tareas a medida que se agregan o eliminan asignaciones de recursos, considerando siempre las fechas asociadas a la tarea correspondiente.

Asimismo, se identificó la necesidad de registrar no solo qué recursos requiere una tarea, sino también cuántas instancias de cada uno. Para resolver esto se introdujo la clase RecursoNecesario, que encapsula el vínculo entre un recurso y la cantidad necesaria. De esta manera, cada tarea mantiene una colección de instancias de RecursoNecesario, permitiendo expresar de forma clara y precisa sus necesidades operativas.

Se asegura una alta cohesión mediante el uso de estas clases auxiliares, RangoDeUso y RecursoNecesario, que encapsulan responsabilidades bien delimitadas y se encuentran acopladas únicamente con una entidad del dominio (Recurso y Tarea, respectivamente). Al mismo tiempo, las clases Recurso y Tarea mantienen sus responsabilidades principales sin ser sobrecargadas funcionalmente. El único cambio significativo en Recurso es la incorporación del control de su propia disponibilidad, lo cual es coherente con su rol dentro del sistema.

En cuanto al tratamiento de los conflictos de disponibilidad, la responsabilidad recae sobre el GestorTareas, dado que este orchestra la lógica de asignación y eliminación de recursos. Esta clase valida si un recurso está disponible en el intervalo temporal requerido por la tarea, consultando los rangos de uso existentes. En caso de conflicto, también se encarga de gestionar los mecanismos de resolución definidos: la búsqueda de recursos alternativos, la reprogramación de la tarea o la asignación forzada del recurso, siguiendo siempre los criterios de integridad y consistencia del sistema.

Cobertura de pruebas unitarias: (Ver anexo 3)

Referencias Bibliográficas

Chinta, M. (2024, 13 de octubre). *EF Core: In-Memory Database Unit testing*. Medium (CodeNx). Recuperado de <https://medium.com/codenx/ef-core-in-memory-database-unit-testing-02d4658a9c78>

Code, B. (2024, 15 de abril). *Getting started with .NET Core Entity Framework Code First, EF Core Introduction, EF Core guide, EF Core Best practices*. Medium. Recuperado de <https://medium.com/@codebob75/entity-framework-core-code-first-introduction-best-practices-repository-pattern-clean-22b6152bcb81>

Dotnet. (s.f.). *Seed InMemory database automatically (i.e. without calling EnsureCreated)* [Issue #11666]. GitHub. Recuperado de <https://github.com/dotnet/efcore/issues/11666>

IngSoft-DA1. (s.f.). *Ejemplos-tecnologia* [Software]. GitHub. Recuperado de <https://github.com/IngSoft-DA1/Ejemplos-tecnologia>

Microsoft. (s.f.). *Entity Framework FAQ - Entity Classes*. Microsoft. Recuperado el 10 de junio de 2025, de https://learn.microsoft.com/en-us/archive/technet-wiki/3820.entity-framework-faq-entity-classes#Does_the_Entity_Framework_require_objects_with_public_empty_constructors

Microsoft. (2023, 30 de marzo). *Introducción a las relaciones – EF Core*. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/ef/core/modeling/relationships>

Microsoft. (s.f.). *Modelado de datos en Entity Framework Core*. Microsoft Learn. Recuperado el 11 de junio de 2025, de <https://learn.microsoft.com/es-es/ef/core/modeling/>

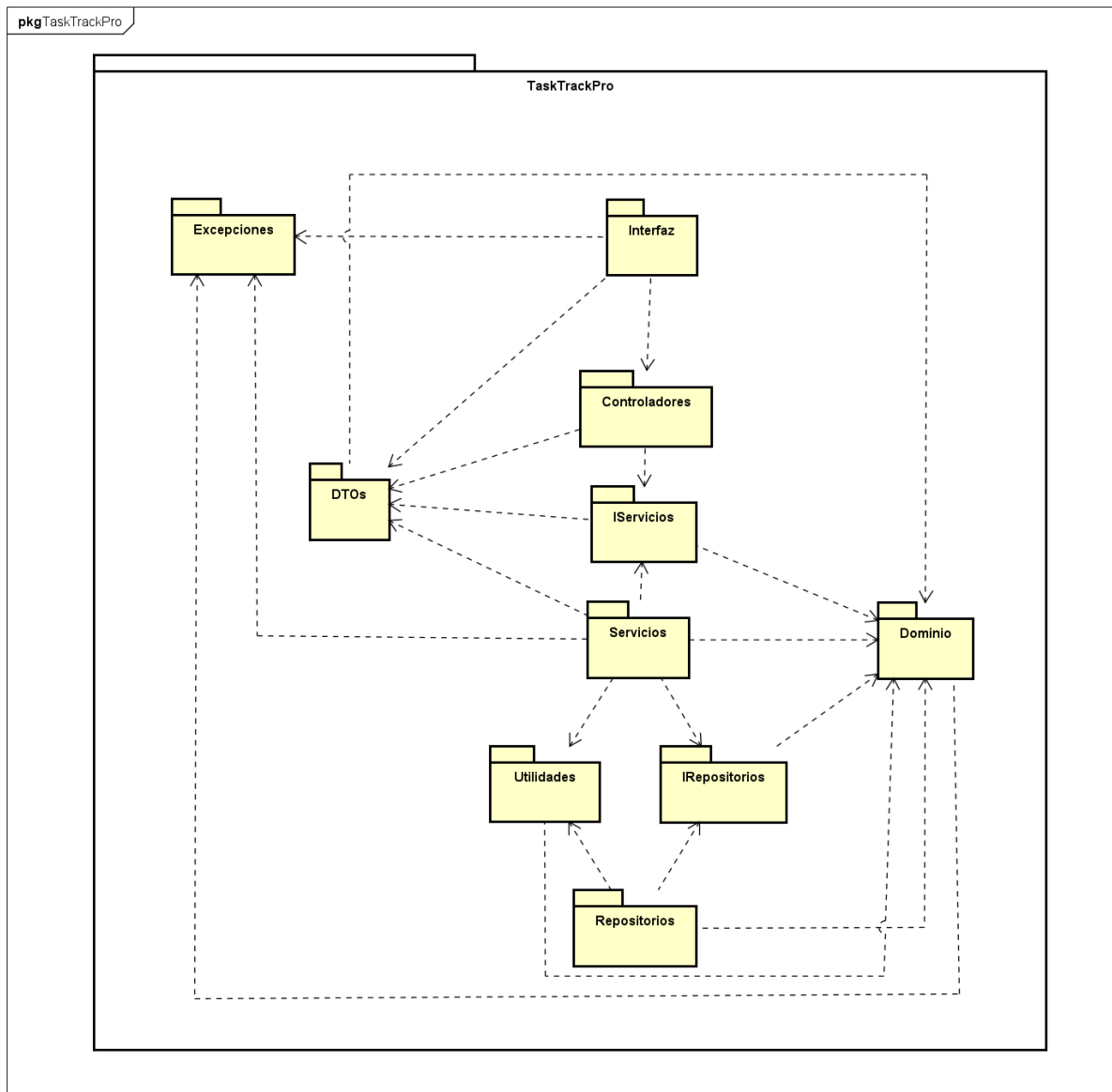
Toumi, A. (2023, 12 de junio). *Data Transfer Objects (DTOs): A Comprehensive Guide*. Medium. Recuperado de <https://abderahmanetoumi.medium.com/data-transfer-objects-dtos-a-comprehensive-guide-2d00e8fa2ec3>

ANEXO 1

Diagramas UML

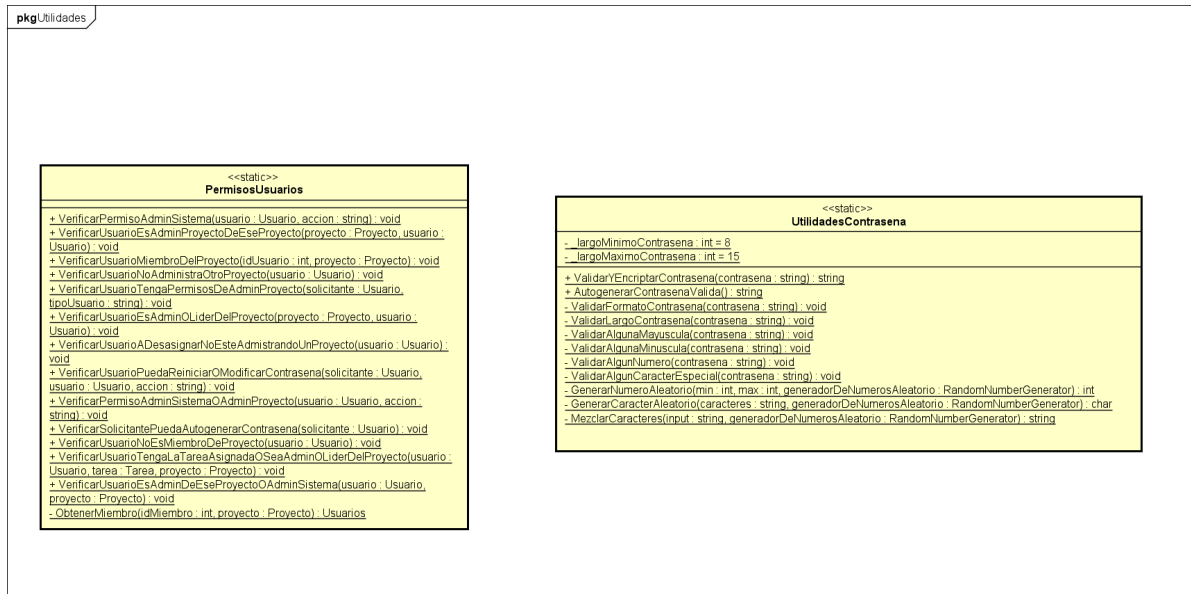
En este anexo se presentan los diagramas UML, en la carpeta entregada están adjuntados los .svg de cada uno de los diagramas.

1.1 Diagrama de paquetes

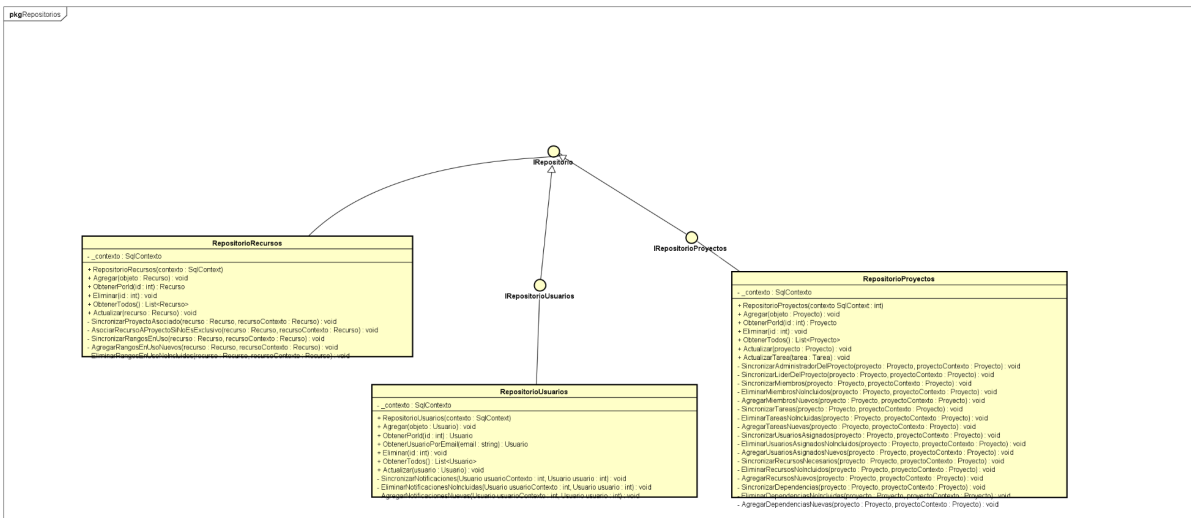


1.2 Dominio

1.4 Utilidades



1.5 Repositorios



ANEXO 2

Diagramas de interacción

En este anexo se presentan fotos de los diagramas de interacción, en la carpeta entregada están adjuntados los .svg de cada uno de los diagramas.

Diagrama 1: Asignar miembro a Proyecto.

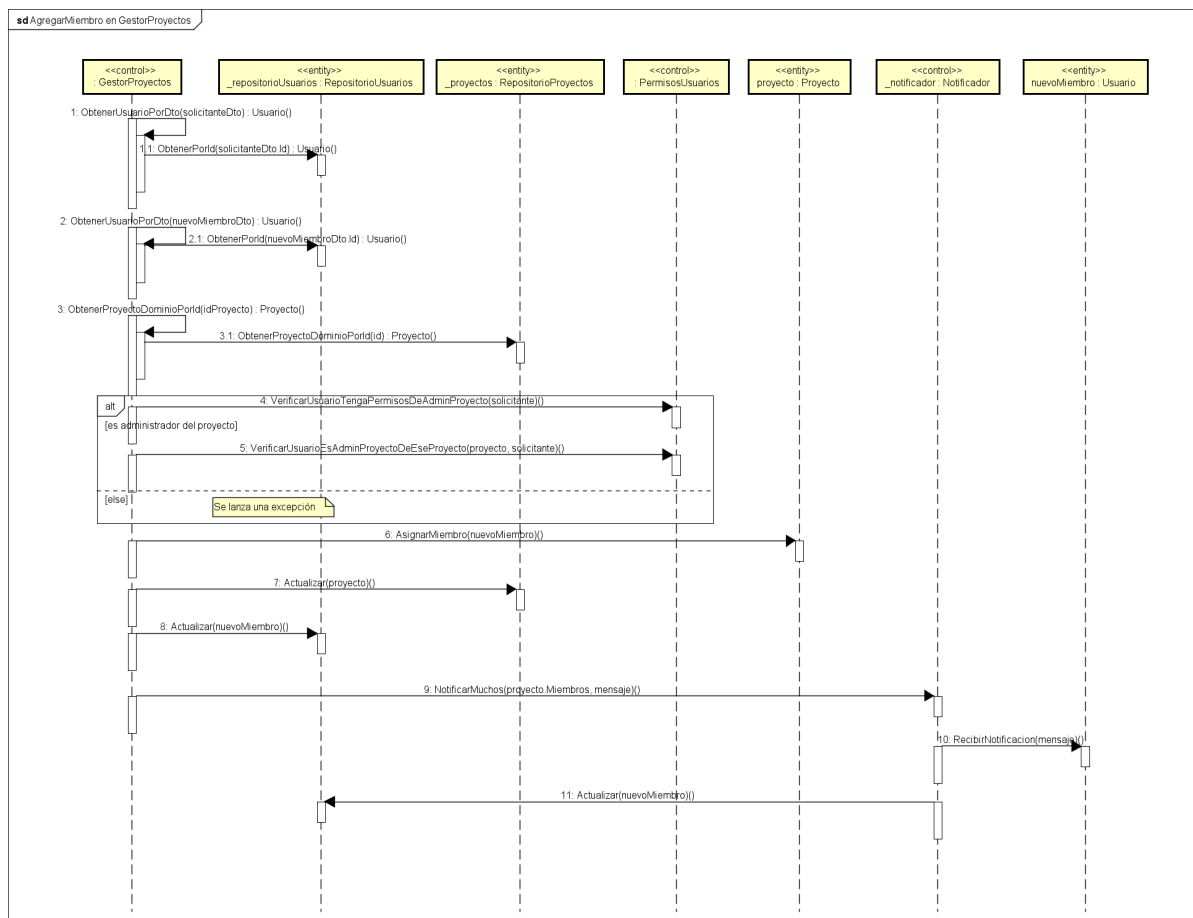


Diagrama 2: Crear Proyecto.

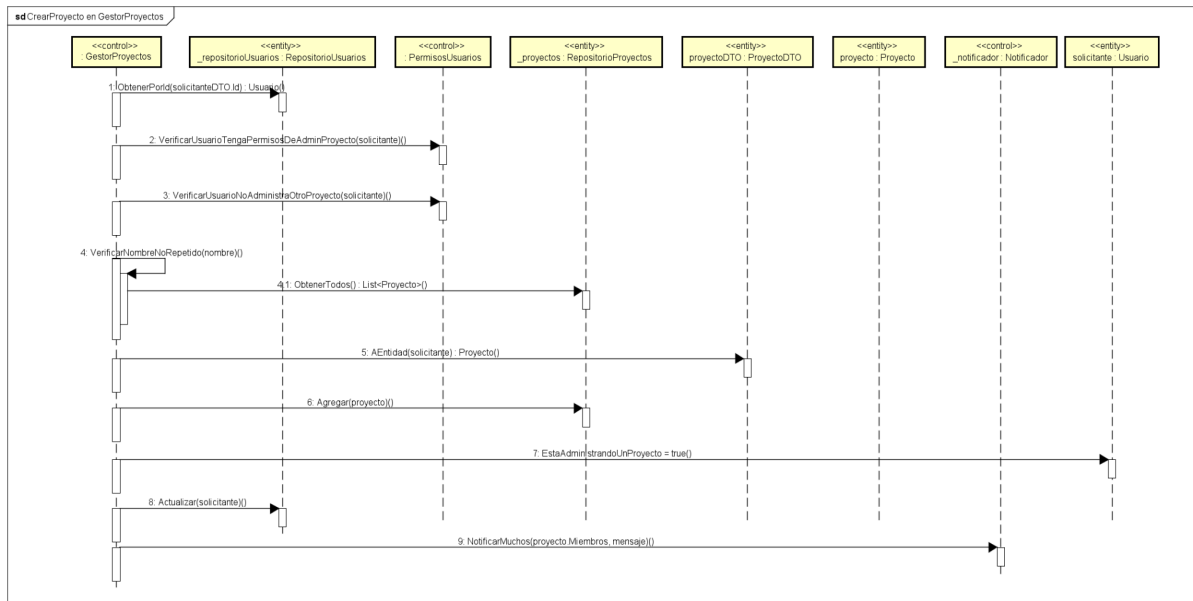
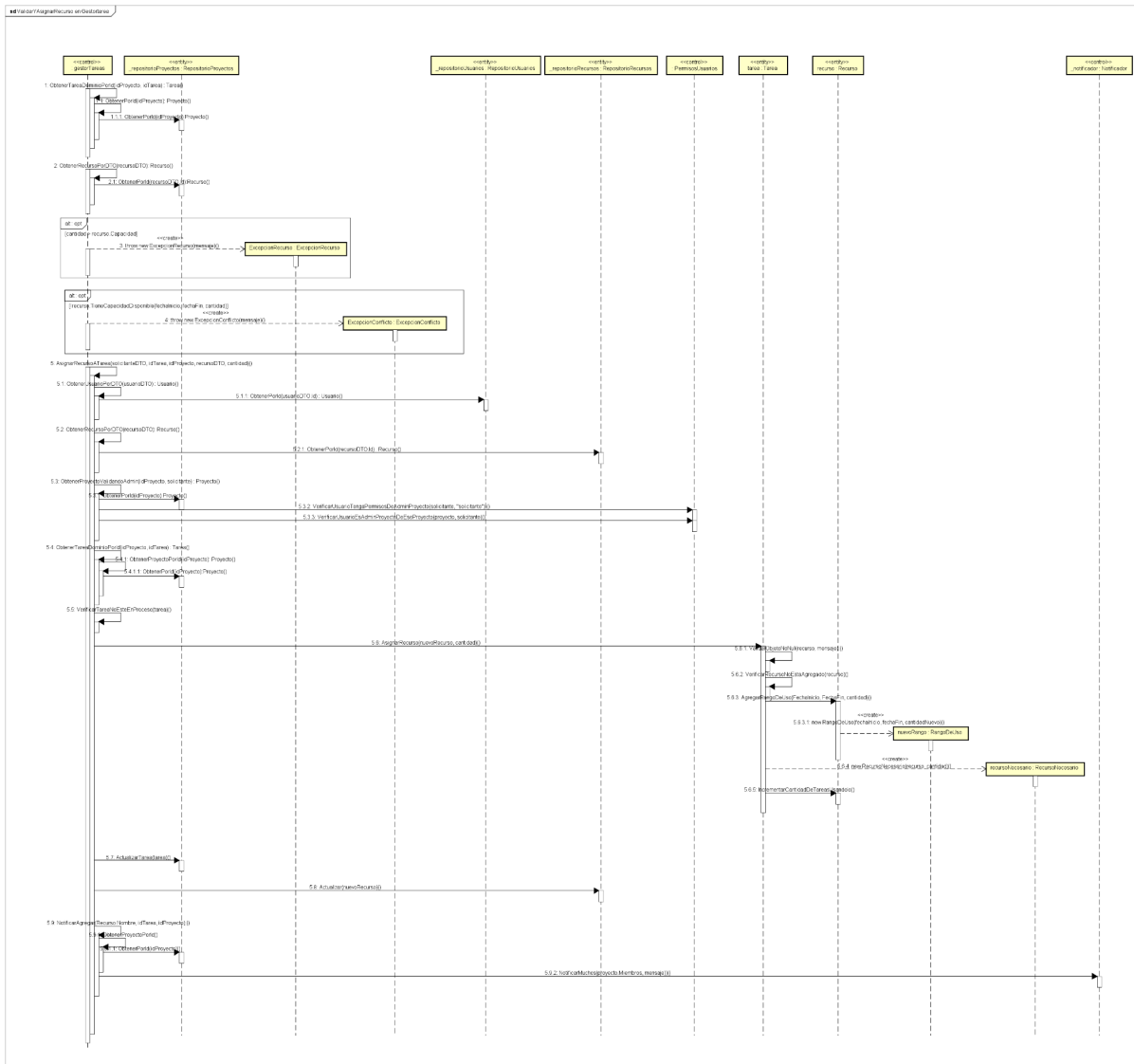


Diagrama 3: ValidarYAsignar Recurso a Tarea.

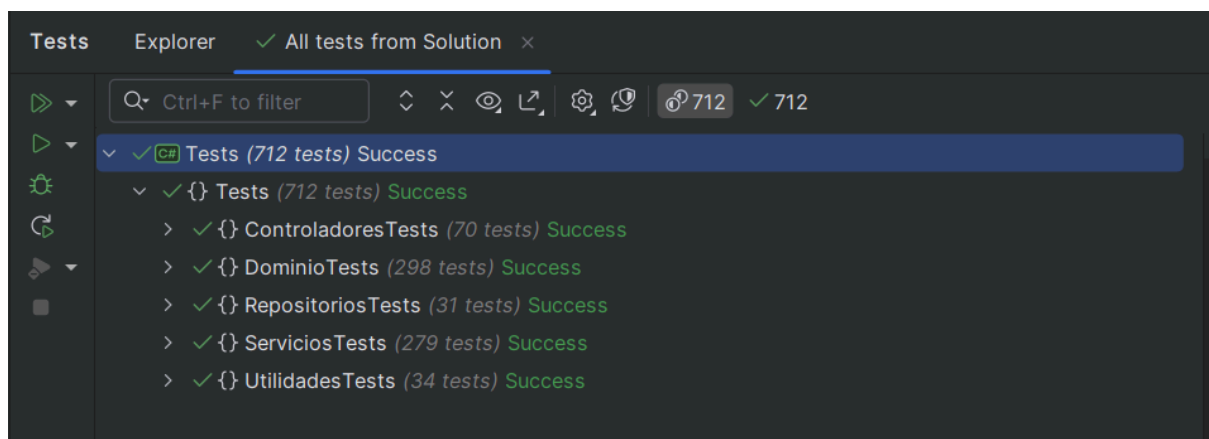


ANEXO 3

Cobertura de pruebas unitarias

En esta sección del anexo se muestran capturas de pantalla de la cobertura de los tests unitarios, en la carpeta entregada está adjuntado el .html de la cobertura.

> [icon] Controladores	100%	0/214
> [icon] Repositorios	100%	0/408
> [icon] Utilidades	100%	0/150
▼ [icon] Servicios	99%	4/1174
▼ {} Servicios	99%	4/1174
> {} Notificaciones	100%	0/38
> {} Exportacion	100%	0/70
> {} CaminoCritico	100%	0/150
▼ {} Gestores	99%	4/916
> [icon] GestorProyectos	100%	0/230
> [icon] GestorRecursos	100%	0/171
> [icon] GestorUsuarios	100%	0/158
> [icon] GestorTareas	99%	4/357
▼ [icon] Dominio	99%	3/1034
▼ {} Dominio	99%	3/1034
> [icon] Dependencia	100%	0/39
> [icon] Notificacion	100%	0/16
> [icon] Proyecto	100%	0/255
> [icon] RangoDeUso	100%	0/36
> [icon] RecursoNecesario	100%	0/34
> [icon] Tarea	100%	0/304
> [icon] Usuario	100%	0/129
> [icon] Recurso	99%	3/221



Aclaraciones sobre tests y cobertura:

La aplicación fue desarrollada siguiendo el enfoque *Test Driven Development* (TDD), por lo que todos los tests implementados pasan correctamente. No obstante, la cobertura de código

no alcanza el 100%. Esto se debe a que, durante el desarrollo de nuevas funcionalidades relacionadas con la gestión de recursos, se realizaron refactorizaciones que introdujeron algunas líneas de código sin pruebas específicas asociadas.

Por limitaciones de tiempo, no se pudo determinar con certeza si esas líneas no cubiertas corresponden a código redundante o simplemente a escenarios aún no testeados. A pesar de ello, se priorizó la cobertura de los caminos críticos y se verificó el correcto funcionamiento de las funcionalidades principales del sistema.

ANEXO 4

Instalación

Todo lo referente a la instalación de la aplicación, se documenta en el archivo README.md, presente en el repositorio así como en la carpeta del release. Se detalla una guía de instalación, que se complementa con lo documentado en el *Anexo 6* sobre el contenido de los scripts entregados y los datos de prueba.

Notas sobre la conexión a la base de datos:

Los strings de conexión utilizados por la aplicación se encuentran configurados en el archivo appsettings.json. Por defecto, se utiliza localhost como servidor de base de datos, al igual que en las conexiones definidas en DBeaver, por lo que no deberían requerir modificación en entornos donde SQL Server esté ejecutándose localmente. Si se utiliza una instancia remota, un nombre distinto al del servidor predeterminado (localhost) o se modifica el puerto, será necesario actualizar el valor del campo "ConnectionStrings:DefaultConnection" en appsettings.json antes de ejecutar la aplicación.

ANEXO 5

Pruebas unitarias

Se utilizaron pruebas automatizadas escritas con el framework MSTest, aplicando la metodología *Test Driven Development* (TDD) en todo el proceso de desarrollo. La evidencia del uso de dicha metodología puede observarse en el historial de commits del repositorio, donde se refleja claramente la secuencia de escritura de pruebas seguida por la implementación de la lógica correspondiente.

Debido a que la lógica del sistema fue construida a partir de pruebas, se asegura una alta cobertura y validación continua de todos los componentes del dominio, desde entidades y servicios hasta operaciones de persistencia y controladores.

Para los casos que involucran acceso a base de datos, se utilizó *EF Core In-Memory Database Provider*, un proveedor de base de datos en memoria que simula el comportamiento de una base de datos relacional sin requerir una instancia física. Esta herramienta permite ejecutar pruebas rápidas y aisladas, sin necesidad de levantar un servidor real, facilitando así la automatización y estabilidad de los tests.

Gracias a esta estrategia, se logró mantener pruebas alineadas con los principios F.I.R.S.T. (*Fast, Independent, Repeatable, Self-validating, Timely*), garantizando que el sistema sea confiable, mantenible y fácilmente extensible, respaldado por una conjunto de pruebas completo y automatizado.

ANEXO 6

Datos de prueba

En el release *Entrega 2*, correspondiente a la entrega de este obligatorio, se incluyen 2 scripts *.sql* para hacer uso de la base de datos configurada. En la guía de instalación (ver anexo 4.), se brindan los pasos para ejecutar la aplicación adecuadamente utilizando dichos scripts.

El primer script entregado es el archivo *scriptCreacion.sql*, el cual contiene las sentencias necesarias para la creación de tablas, sin ningún tipo de dato precargado, a excepción del Administrador Inicial, usuario que permite ingresar al sistema por primera vez. Las credenciales de dicho usuario son las siguientes:

Email: admin@sistema.com

Contraseña: TaskTrackPro@2025

Adicionalmente, se entrega un script con los datos de prueba requeridos, el cual contiene las sentencias necesarias para insertar los datos en las tablas creadas con el script de creación. Estos datos de prueba incluyen los 10 proyectos solicitados, con sus 10 administradores, 6 recursos generales, algunos recursos exclusivos de algunos proyectos, 20 tareas con dependencias (2 tareas por proyecto, una dependiente de la otra), 3 líderes de proyectos (cada uno de un solo proyecto diferente), 10 usuarios comunes (sin roles), con 3 de ellos que son miembros de todos los proyectos. Además, la mayoría de los usuarios tienen algún tipo de alertas (notificaciones, visibles en la ventana *Notificaciones*), ya que muchos de los eventos del sistema simulados para generar los datos de prueba, implican notificaciones.

Todos los usuarios del sistema, tienen su email correspondiente pero comparten la misma contraseña: Admin123\$

Además, se evaluó qué tarea se agregaría en base a los datos de prueba dados, para probar el algoritmo para la resolución de conflicto de recursos. Se recomienda agregar la siguiente tarea al proyecto *Diseño de aplicaciones 1*:

- Título: Estudiar para el parcial
- Descripción: Estudiar para el parcial
- Duración en días: 5

- Fecha de inicio: 25/06/2025
- Dependencias:
 - Dependencia SS con la tarea *Obligatorio 2*
- Recursos necesarios:
 - *ChatGPT Pro* de tipo *IA*, con cantidad 1

Al asignar este último recurso, debería aparecer un conflicto ya que la tarea *Obligatorio 2* hace uso del mismo, que solo tiene capacidad 1. A su vez, se deben mostrar los botones de resolución de conflictos, que gestionan la resolución mediante envíos de notificaciones.

Para realizar esta prueba, se recomienda ingresar como un usuario administrador de ese proyecto (el administrador del proyecto *Diseño de aplicaciones 1* tiene email profesda1@gmail.com).